

비전공자를 위한 실전 가이드

My-Gym 맞춤형 운동 코치 AI 만들기

오픈소스 AI로 내 리눅스 서버PC에
나만의 AI 운동 코치를 구축하는 전 과정

대상 환경 : 리눅스 서버PC (NVIDIA GT720)
난이도 : 입문 (코딩 경험 불필요, 복사·붙여넣기로 진행)
2026년 7월 작성

목차

1. 무엇을 만드는 건가요? — 전체 그림
2. 시작 전 준비 — 내 PC 사양 확인하기
3. 1단계 : 서버PC에 Ollama 설치하고 AI 실행하기
4. 2단계 : 운동 코치 학습 데이터 만들기 (가장 중요!)
5. 3단계 : 구글 Colab에서 무료로 모델 교육(파인튜닝)하기
6. 4단계 : 완성된 모델을 서버PC에 설치하기
7. 5단계 : My-Gym과 연결하기 (API)
8. 문제 해결 FAQ
9. 용어 사전

1. 무엇을 만드는 건가요? — 전체 그림

이 가이드를 끝내면, **My-Gym**에서 질문하면 여러분이 직접 가르친 운동 코치 AI가 답해주는 시스템이 완성됩니다. 인터넷 없이도 서버PC 안에서 동작하고, 사용료도 없습니다.



작업은 크게 두 곳에서 나뉘서 합니다

여기서 꼭 이해해야 할 핵심이 하나 있습니다. AI 작업은 "**실행**"과 "**교육(학습)**"으로 나뉘는데, 필요한 컴퓨터 성능이 완전히 다릅니다.

구분	하는 일	어디서?	이유
실행 (추론)	완성된 AI에게 질문하고 답을 받음	서버PC	CPU만으로도 소형 모델 실행 가능
교육 (파인튜닝)	AI에게 운동 코치 역할을 가르침	구글 Colab (무료)	고성능 GPU가 필요한데 구글이 무료로 빌려줌

⚠ GT720 그래픽카드에 대한 솔직한 안내

GT720은 2014년에 나온 사무용 그래픽카드로, 메모리(VRAM)가 1~2GB뿐이고 최신 AI 소프트웨어(CUDA)가 지원을 중단한 세대입니다. 따라서 **AI 작업에서는 없는 것과 마찬가지**입니다. 하지만 걱정하지 마세요. 이 가이드는 처음부터 "실행은 CPU로, 교육은 구글 무료 GPU로" 하도록 설계되어 있어서 GT720 이어도 전혀 문제없이 완주할 수 있습니다.

전체 5단계 로드맵

단계	내용	예상 소요 시간
1단계	서브PC에 Ollama 설치, 기본 AI 실행해보기	30분
2단계	운동 코치 학습 데이터(질문-답변) 만들기	수일 (품질이 생명)
3단계	구글 Colab에서 무료 파인튜닝	1~2시간
4단계	완성 모델을 서브PC의 Ollama에 등록	30분
5단계	My-Gym에서 API로 호출해 연결	1~2시간

2. 시작 전 준비 — 내 PC 사양 확인하기

터미널(검은 명령어 창)을 엽니다. 대부분의 리눅스에서 **Ctrl + Alt + T** 를 누르면 열립니다. 아래 명령어를 한 줄씩 입력하고 Enter를 누르세요.

```
# 메모리(RAM) 용량 확인 — "Mem:" 줄의 첫 숫자가 전체 용량
free -h

# CPU 정보 확인
lscpu | grep "Model name"

# 디스크 남은 공간 확인 — 최소 20GB 여유 권장
df -h /
```

항목	최소	권장	비고
RAM	8GB	16GB	8GB면 3B 모델, 16GB면 7B 모델까지 실행 가능
디스크 여유	10GB	20GB 이상	모델 파일이 2~5GB씩 차지
GPU	불필요	—	GT720은 사용하지 않음 (CPU로 실행)
인터넷	필요	—	모델 다운로드와 Colab 교육 시에만 필요

i "3B, 7B"가 뭔가요?

AI 모델의 크기 단위입니다. B는 Billion(10억)으로, 모델이 가진 지식 매개변수(파라미터) 개수를 뜻합니다. 숫자가 클수록 똑똑하지만 무겁습니다. CPU만 있는 PC에서는 3B(30억)급이 속도와 품질의 균형이 가장 좋습니다.

3. 1단계 : 서버PC에 Ollama 설치하고 AI 실행하기

Ollama(올라마)는 오픈소스 AI 모델을 내 컴퓨터에서 쉽게 실행해주는 무료 프로그램입니다. "AI 실행기"라고 생각하면 됩니다.

1 Ollama 설치

터미널에 아래 한 줄을 붙여넣고 Enter. 관리자 비밀번호를 물으면 입력하세요.

```
curl -fsSL https://ollama.com/install.sh | sh
```

설치 확인:

```
ollama --version  
# "ollama version 0.x.x" 처럼 버전이 나오면 성공
```

2 기본 모델 다운로드 및 첫 대화

운동 코치의 바탕이 될 모델을 받아 실행합니다. 한국어 성능이 좋은 소형 모델로 **Qwen2.5 3B**를 추천합니다.

```
ollama run qwen2.5:3b
```

처음 한 번은 약 2GB를 다운로드하므로 몇 분 걸립니다. 끝나면 **>>>** 프롬프트가 나타나는데, 여기에 한국어로 질문해보세요.

```
>>> 벤치프레스 할 때 어깨가 아픈 이유가 뭐야?  
(AI가 답변을 생성합니다)
```

```
>>> /bye      ← 대화 종료 명령
```

✓ 체크포인트

AI가 한국어로 답하면 1단계 성공입니다. 답변 속도가 느려도 정상입니다(CPU 실행이라 초당 몇 단어 수준). 답이 아직 "일반적인" 수준인 것도 정상입니다 — 이제부터 My-Gym 맞춤으로 교육시킬 거니까요.

4. 2단계 : 운동 코치 학습 데이터 만들기 (가장 중요!)

AI 교육의 성패는 코드가 아니라 **데이터 품질**이 결정합니다. 전체 시간의 80%를 여기에 쓴다고 생각하세요.

어떤 데이터를 만드나요?

"이런 질문이 오면 → 이렇게 답해라"라는 **질문-답변 쌍**을 만듭니다. My-Gym 사용자가 실제로 물어볼 만한 질문과, 여러분이 원하는 이상적인 답변을 짝지으면 됩니다.

분량	기대 효과
100~300쌍	말투·답변 형식이 바뀌기 시작 (최소 시작점)
300~1,000쌍	운동 코치다운 일관된 답변 (권장 목표)
1,000쌍 이상	세부 상황까지 안정적으로 대응

데이터 파일 형식 (JSONL)

JSONL은 한 줄에 하나의 질문-답변을 적는 텍스트 파일입니다. 메모장으로도 만들 수 있습니다. 파일 이름은 `my_gym_data.jsonl` 로 저장하세요.

```
{"instruction": "3개월차 초보자인데 3분할 루틴 추천해줘", "output": "3개월차라면 아직 3분할보다 무분할 또는 2분할이 효과적이에요. 그래도 3분할을 원하신다면 이렇게 추천드려요.\n\n[1일차 - 가슴/삼두] 벤치프레스 4세트, 인클라인 덤벨프레스 3세트...\n\n각 부위별 주 2회 자극이 초보자에게 더 좋으니, 4주 후 무분할과 비교해보는 걸 권해요."}\n\n{"instruction": "다이어트 중인데 운동 전에 뭘 먹어야 해?", "output": "다이어트 중이라도 공복 운동은 추천하지 않아요. 운동 1시간 전에 바나나 1개나 고구마 반 개처럼 소화 빠른 탄수화물을 100~150kcal 정도 섭취하면 운동 강도를 유지하면서 지방 연소 효과도 챙길 수 있어요."}\n\n{"instruction": "스쿼트하면 무릎에서 소리가 나", "output": "통증 없이 소리만 나는 건 대부분 괜찮지만, 통증이 함께 있다면 즉시 중단하고 전문의 상담을 받으세요. 통증이 없다면 힙힌지 자세 점검과 충분한 워밍업(맨몸 스쿼트 2세트)을 먼저 해보시길 권해요."}
```

⚠ 형식 규칙 (틀리면 교육이 실패합니다)

- ① 한 줄에 하나의 {...}만 — 줄바꿈으로만 구분
- ② 따옴표는 반드시 큰따옴표(") 사용
- ③ 답변 안에서 줄을 바꾸고 싶으면 실제 줄바꿈 대신 `\n` 입력
- ④ 파일 인코딩은 UTF-8 (리눅스 기본값이라 보통 신경 안 써도 됨)

좋은 데이터를 만드는 5가지 요령

1. **실제 질문에서 출발** — My-Gym에 쌓인 실제 사용자 질문, 헬스 커뮤니티의 자주 묻는 질문을 수집하세요.

2. **답변에 "내 코치 스타일"을 담기** — 말투(존댓말/친근함), 답변 구조(결론 먼저 → 이유 → 실행법), 길이를 모든 답변에 일관되게 적용하세요. AI는 이 패턴을 배웁니다.
3. **주제를 골고루** — 운동 루틴, 자세 교정, 식단, 부상 대처, 동기부여 등 실제 질문 비율대로 섞으세요.
4. **안전 답변 반드시 포함** — "통증이 있다", "약을 먹는 중이다" 같은 질문에는 "전문의 상담 권장"으로 답하는 예시를 30개 이상 넣으세요. 안전과 직결됩니다.
5. **AI로 초안 만들고 직접 검수** — Claude 같은 AI에게 "헬스 초보 질문 50개와 코치 답변을 JSONL로 만들어 줘"라고 시켜 초안을 얻고, **내용의 정확성은 반드시 직접 확인**하세요. 잘못된 데이터로 교육하면 잘못 답하는 AI가 됩니다.

형식 검사하기

파일을 다 만들면 서브PC 터미널에서 형식 오류를 검사하세요.

```
python3 -c "  
import json  
ok = 0  
for i, line in enumerate(open('my_gym_data.jsonl', encoding='utf-8'), 1):  
    if not line.strip(): continue  
    try:  
        d = json.loads(line)  
        assert 'instruction' in d and 'output' in d  
        ok += 1  
    except Exception as e:  
        print(f'{i}번째 줄 오류: {e}')  
print(f'정상 데이터: {ok}개')  
"
```

5. 3단계 : 구글 Colab에서 무료로 모델 교육하기

구글 Colab은 구글이 고성능 GPU 컴퓨터를 브라우저에서 무료로 빌려주는 서비스입니다. **파인튜닝**(미세조정)이란 이미 똑똑한 기존 모델에 내 데이터로 "추가 교육"을 하는 것으로, 여기서는 **Unsloth(언슬로스)**라는 도구를 씁니다. 무료 GPU에서도 돌아갈 만큼 효율적이라 입문자에게 가장 인기 있는 도구입니다.

i LoRA(로라)란?

모델 전체(수십억 개 파라미터)를 다시 교육하는 대신, 작은 "어댑터"만 붙여서 교육하는 기법입니다. 비유하면 교과서 전체를 다시 쓰는 게 아니라 포스트잇 메모를 붙이는 방식이라, 무료 GPU로도 1~2시간이면 끝납니다.

1 Colab 열고 GPU 켜기

1. 브라우저에서 **colab.research.google.com** 접속 → 구글 계정 로그인

2. [새 노트] 클릭

3. 상단 메뉴 [런타임] → [런타임 유형 변경] → **T4 GPU** 선택 → 저장

2 데이터 업로드

왼쪽 폴더 아이콘(📁) 클릭 → 업로드 버튼으로 `my_gym_data.jsonl` 을 올립니다.

3 교육 코드 실행

아래 코드를 셀에 붙여넣고 실행(▶ 버튼 또는 Shift+Enter)합니다. 셀은 총 4개로 나눠서 순서대로 실행하세요.

셀 1 — 도구 설치 (약 3분)

```
!pip install unsloth -q
```

셀 2 — 기본 모델 불러오기

```
from unsloth import FastLanguageModel

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Qwen2.5-3B-Instruct", # 1단계에서 쓴 것과 같은 계열
    max_seq_length = 2048, # 한 번에 처리할 최대 글자 길이
    load_in_4bit = True, # 메모리 절약 모드 (무료 GPU 필수)
)

# LoRA 어댑터 부착 (여기만 교육됨)
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # 어댑터 크기 (16이 무난한 기본값)
    lora_alpha = 16,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"],
)
```

셀 3 — 내 데이터 연결

```

from datasets import load_dataset

dataset = load_dataset("json", data_files="my_gym_data.jsonl", split="train")

# 모델이 이해하는 대화 형식으로 변환
def to_chat(example):
    messages = [
        {"role": "system", "content": "당신은 My-Gym의 전문 운동 코치입니다. 친절하고 안전을 최우선으로 하\n는 조언을 합니다."},
        {"role": "user", "content": example["instruction"]},
        {"role": "assistant", "content": example["output"]},
    ]
    return {"text": tokenizer.apply_chat_template(messages, tokenize=False)}

dataset = dataset.map(to_chat)
print(f"교육 데이터 {len(dataset)}건 준비 완료")

```

셀 4 — 교육 시작 (데이터 500건 기준 약 30분~1시간)

```

from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = dataset,
    dataset_text_field = "text",
    max_seq_length = 2048,
    args = TrainingArguments(
        per_device_train_batch_size = 2,
        gradient_accumulation_steps = 4,
        num_train_epochs = 3,          # 데이터 전체를 3바퀴 학습
        learning_rate = 2e-4,         # 학습 속도 (기본값 권장)
        fp16 = True,
        logging_steps = 10,
        output_dir = "outputs",
    ),
)
trainer.train()

```

실행 중 **loss** 라는 숫자가 계속 표시됩니다. 이 숫자가 **점점 줄어들면 교육이 잘 되고 있는 것**입니다 (예: 2.1 → 1.3 → 0.9).

4 결과를 GGUF 파일로 내보내기

GGUF는 CPU에서 실행하기 좋게 압축한 모델 파일 형식입니다. 서버PC의 Ollama가 이 형식을 사용합니다.


```
model.save_pretrained_gguf(
    "my_gym_coach",
    tokenizer,
    quantization_method = "q4_k_m", # 품질과 용량의 균형점 (약 2GB)
)
```

완료되면 왼쪽 파일 목록의 **my_gym_coach** 폴더 안에 **.gguf** 파일이 생깁니다. 파일에 마우스 우클릭 → [다운로드]로 서버PC에 저장하세요.

⚠ Colab 무료 버전 주의사항

- ① 90분간 조작이 없으면 연결이 끊길 수 있으니 교육 중에는 가끔 화면을 클릭해주세요.
- ② 연결이 끊기면 파일이 사라집니다. GGUF 파일이 생기면 **바로 다운로드**하세요.
- ③ 하루 GPU 사용량에 한도가 있습니다. 한도에 걸리면 다음 날 다시 하면 됩니다.

6. 4단계 : 완성된 모델을 서버PC에 설치하기

1 모델 파일 배치

다운로드한 GGUF 파일을 홈 폴더에 옮기고 이름을 정리합니다.

```
mkdir -p ~/my-gym-ai
mv ~/Downloads/*.gguf ~/my-gym-ai/my_gym_coach.gguf
```

2 Modelfile 만들기

Modelfile은 "이 모델을 이런 설정으로 실행해라"라고 Ollama에 알려주는 설정 파일입니다.

```
cat > ~/my-gym-ai/Modelfile << 'EOF'
FROM ./my_gym_coach.gguf
```

```
SYSTEM ""당신은 My-Gym의 전문 운동 코치입니다. 친절하고 안전을 최우선으로 하는 조언을 합니다. 통증이나 부상
관련 질문에는 반드시 전문의 상담을 권합니다.""
```

```
PARAMETER temperature 0.7
PARAMETER num_ctx 2048
EOF
```

i temperature란?

답변의 창의성 조절 값입니다. 0에 가까울수록 일관되고 딱딱한 답, 1에 가까울수록 다양하고 자유로운 답이 나옵니다. 코치 역할에는 0.5~0.7이 적당합니다.

3 Ollama에 등록하고 테스트

```
cd ~/my-gym-ai
ollama create my-gym-coach -f Modelfile

# 실행!
ollama run my-gym-coach
>>> 헬스 처음인데 뭐부터 하면 돼?
```

✓ 체크포인트 — 교육 전후 비교

같은 질문을 기본 모델(`ollama run qwen2.5:3b`)과 내 모델(`ollama run my-gym-coach`)에 던져보세요. 내 모델이 2단계에서 정한 말투와 답변 구조를 따라 하고 있다면 성공입니다. 학습에 쓰지 않은 새 질문 20개 정도로 테스트하고, 이상한 답이 많으면 해당 주제의 데이터를 보강해 3단계를 다시 하면 됩니다.

7. 5단계 : My-Gym과 연결하기 (API)

Ollama는 설치만 하면 자동으로 로컬 API 서버가 켜집니다 (주소: <http://localhost:11434>). My-Gym에서 이 주소로 질문을 보내면 AI의 답변을 받을 수 있습니다.

먼저 터미널에서 테스트

```
curl http://localhost:11434/api/chat -d '{
  "model": "my-gym-coach",
  "messages": [
    {"role": "user", "content": "어깨 운동 루틴 추천해줘"}
  ],
  "stream": false
}'
```

Python으로 연결하는 예시

My-Gym이 Python 기반이라면 아래 함수를 그대로 쓸 수 있습니다.

```
import requests

def ask_coach(question: str) -> str:
    """My-Gym 운동 코치 AI에게 질문하고 답변을 받는다."""
    res = requests.post(
        "http://localhost:11434/api/chat",
        json={
            "model": "my-gym-coach",
            "messages": [{"role": "user", "content": question}],
            "stream": False,
        },
        timeout=120, # CPU 실행이라 여유 있게
    )
    return res.json()["message"]["content"]

# 사용 예
print(ask_coach("다이어트 중 단백질은 하루에 얼마나 먹어야 해?"))
```

사용자의 운동 기록을 함께 넘기기 (맞춤 답변의 핵심)

질문 앞에 My-Gym에 저장된 사용자 정보를 붙여 보내면, 모델을 다시 교육하지 않아도 **개인 맞춤 답변**이 됩니다.

```
user_context = """[사용자 정보]
- 운동 경력: 6개월 / 주 3회
- 최근 기록: 스쿼트 80kg 5회, 벤치 60kg 5회
- 목표: 체지방 감량"""

question = user_context + "\n\n[질문]\n다음 주 루틴 짜줘"
print(ask_coach(question))
```

✓ 다른 기기에서 접속하려면

My-Gym이 다른 컴퓨터나 스마트폰에서 돌아간다면, 서버PC에서 `sudo systemctl edit ollama` 로 편집 창을 열어 아래 두 줄을 추가하고 `sudo systemctl restart ollama` 로 재시작하세요. 이후 `localhost` 대신 서버PC의 IP 주소로 접속하면 됩니다.

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

단, 같은 공유기(내부망) 안에서만 쓰고, 외부 인터넷에 그대로 노출하지 않도록 주의하세요.

8. 문제 해결 FAQ

증상	원인과 해결
답변이 너무 느리다	CPU 실행이라 초당 몇 단어가 정상. 더 빠르길 원하면 더 작은 모델(qwen2.5:1.5b)로 3단계부터 다시 하거나, RAM 증설/PC 업그레이드 고려.
<code>ollama run</code> 시 메모리 부족 오류	다른 프로그램을 종료하고 재시도. 계속되면 더 작은 모델 사용.
Colab에서 GPU 메모리 부족(OOM)	셀 4의 <code>per_device_train_batch_size</code> 를 1로 낮추기. [런타임]→[세션 다시 시작] 후 재실행.
교육 후에도 답변이 그대로다	데이터가 너무 적거나(300건 미만) 형식이 제각각인 경우. 데이터를 늘리고 말투를 통일한 뒤 <code>num_train_epochs</code> 를 4~5로 올려 재교육.
같은 말만 반복하거나 이상한 답	과적합(데이터를 통째로 외움). <code>num_train_epochs</code> 를 2로 낮추거나 데이터 다양성을 늘려 재교육.
JSONL 오류로 셀 3 실패	4장 끝의 형식 검사 스크립트로 오류 줄을 찾아 수정. 대부분 따옴표나 줄바꿈(\n) 문제.
API 호출 시 연결 거부	<code>systemctl status ollama</code> 로 서비스 확인, 꺼져 있으면 <code>sudo systemctl start ollama</code> .

9. 용어 사전

용어	쉬운 설명
오픈소스 AI	누구나 무료로 내려받아 자기 컴퓨터에서 쓰고 고칠 수 있게 공개된 AI 모델
파라미터 (3B, 7B)	모델의 지식 크기. B=10억 개. 클수록 똑똑하지만 무거움
추론 (실행)	완성된 AI에게 질문에 답을 받는 것. CPU로도 가능
파인튜닝 (미세조정)	기존 모델에 내 데이터로 추가 교육을 하는 것
LoRA	모델 전체가 아닌 작은 어댑터만 교육하는 효율적 기법
Ollama	내 컴퓨터에서 AI 모델을 쉽게 실행해주는 무료 프로그램
Unsloth	무료 GPU에서도 파인튜닝이 가능하게 해주는 효율화 도구
Google Colab	구글이 브라우저에서 GPU 컴퓨터를 무료로 빌려주는 서비스
JSONL	한 줄에 하나씩 데이터를 적는 텍스트 파일 형식
GGUF	CPU 실행에 최적화된 압축 모델 파일 형식
양자화 (q4_k_m)	모델을 압축해 용량과 메모리를 줄이는 기술
과적합	학습 데이터를 통째로 외워버려 새 질문에 약해지는 현상
API	프로그램끼리 대화하는 창구. My-Gym↔AI 연결 통로
temperature	답변의 창의성 조절 값 (0=일관됨, 1=자유로움)

마치며

이 가이드의 핵심 순서를 다시 정리하면: **Ollama 설치 → 데이터 만들기 → Colab에서 교육 → GGUF로 가져오기 → API 연결**입니다. 첫 교육은 데이터 100건으로 작게 시작해서 전체 과정을 한 번 완주해보고, 그다음 데이터를 늘려가며 품질을 올리는 것을 추천합니다. 즐거운 AI 코치 만들기 되세요! ☑